§2 Mechanistic Interpretability — Reproductions

Summary

2.1 Induction heads. On repeated random tokens, GPT-2 small's loss falls from 12.4 to 0.18 nats on the second copy. The five heads with the highest induction score are exactly the five reported in the literature. Patching those five heads together into a corrupted run recovers 81% of the loss gap; no single head recovers more than 14%. A decomposition of their inputs separates **where** they look from **what** they copy. Restoring only the previous-token heads puts the attention back but makes the prediction worse: the heads copy corrupted tokens. The token identity they copy comes mostly through MLP 0. **2.2 Superposition.** Toy ReLU models reproduce the phase change from "represent the most important features" to "pack features into geometric superposition" as sparsity rises.

1 Induction heads in GPT-2 small (card 2.1)

Reproduces Olsson et al. (2022), *In-context learning and induction heads*. Code: `experiments/01_mech_interp/01_induction_heads.py` (about 3 min on CPU).

1.1 Task and metrics

Draw $L = 50$ random tokens $a_1, ..., a_L$ and feed $t = (\text{BOS}, a_1, ..., a_L, a_1, ..., a_L)$, a batch of 8 sequences. On the first copy nothing is predictable. On the second copy, every token is predictable by the **induction rule**: at position q, find where the current token t_q occurred before, and predict the token that followed it. With $A^h \in \mathbb{R}^{T\times T}$ the attention pattern of head h, define

$$\text{ind}(h) = \frac{1}{L}\sum_{q=L+1}^{2L} A^h_{q,\,q-L+1}, \quad \text{prev}(h) = \frac{1}{2L}\sum_{q=1}^{2L} A^h_{q,\,q-1}. \tag{1}$$

$\text{ind}(h)$ is the attention a head pays to the token **after** the earlier occurrence, which is the induction target. $\text{prev}(h)$ is the attention a head pays to the immediately preceding token.

For causal evidence we use **activation patching**. A **corrupted** input t' replaces the first copy with fresh random tokens $b_1, ..., b_L$, so the second copy is no longer a repeat. The metric is the mean log-probability of the correct next token inside the second copy,

$$M(t) = \frac{1}{L-1}\sum_{q=L+1}^{2L-1} \log p_\theta\big(t_{q+1} \mid t_{\le q}\big), \tag{2}$$

with $M_{\text{clean}} = -0.18$ and $M_{\text{corr}} = -11.56$. Patching replaces an activation in the corrupted run with its value from the clean run, and we report the **fraction of the gap recovered**,

$$r = \frac{M_{\text{patched}} - M_{\text{corr}}}{M_{\text{clean}} - M_{\text{corr}}}, \tag{3}$$

so $r = 1$ means the patched component carries everything needed and $r = 0$ means nothing.

1.2 Why two heads are needed: composition

An attention head computes $A_{qk} = \text{softmax}_k\left(x_q W_Q W_K^\top x_k^\top / \sqrt{d_{\text{head}}}\right)$ and writes $\sum_k A_{qk} x_k W_V W_O$ into the residual stream. The induction mechanism is two heads in series:

1. A **previous-token head** h' at position k attends to $k-1$ and writes $e(t_{k-1}) W_V^{h'} W_O^{h'}$ into x_k: "the token before me was t_{k-1}".
2. An **induction head** h at query q has a QK circuit that matches its own token $e(t_q)$ against that written information, so the score is high exactly when $t_{k-1} = t_q$. This is **K-composition**: the key side reads another head's output. Its OV circuit then copies the token t_k found at that key into the logits.

Both halves have to come from the first copy: the **where** (previous-token information in the keys) and the **what** (the identity of t_k in the values). The last experiment below separates them.

1.3 Results

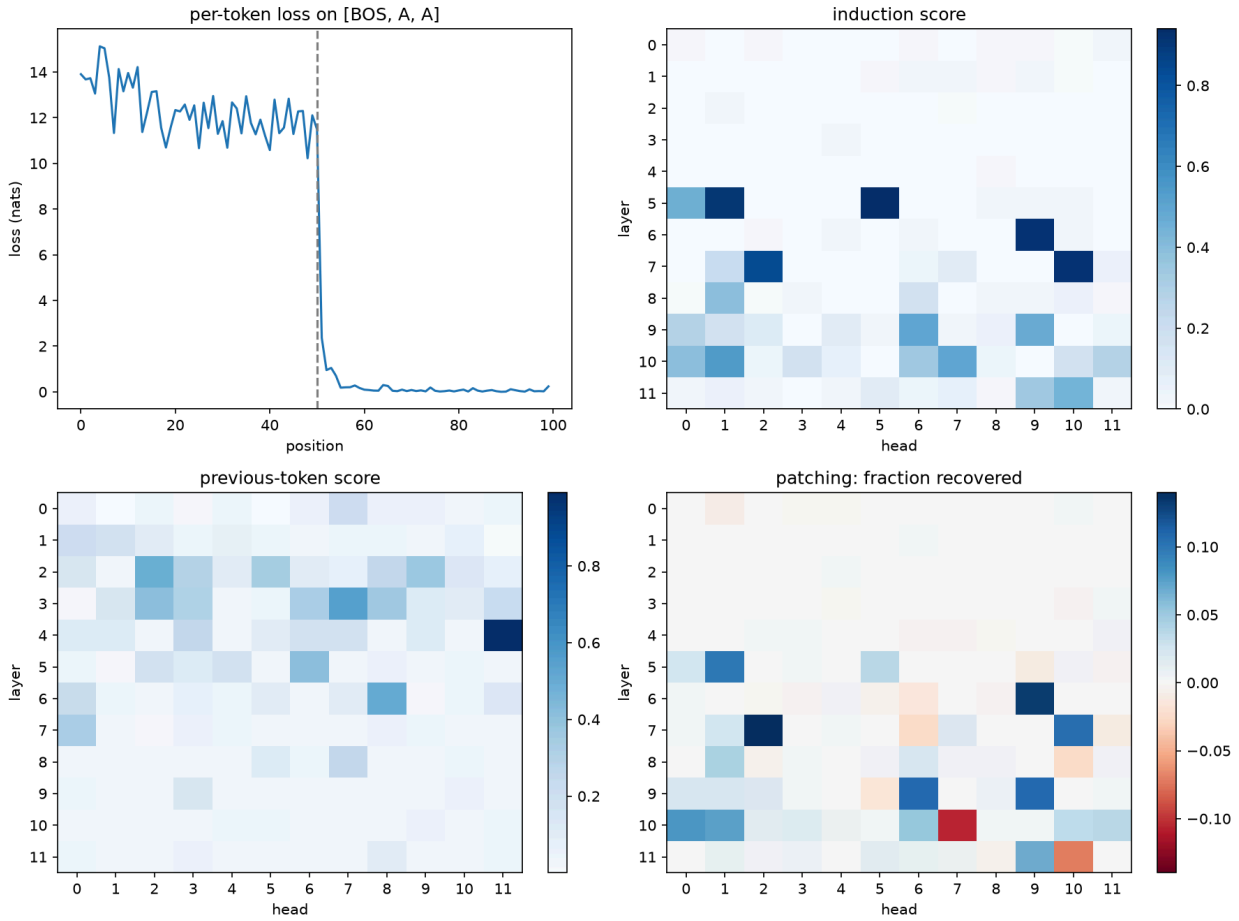


Figure 1: Top left: per-token loss (the dashed line marks the start of the second copy). Top right: induction score. Bottom left: previous-token score. Bottom right: fraction of the gap recovered by patching each head's output (z) alone at all positions.

Behaviour. The loss is 12.36 nats on the first copy (random tokens) and 0.18 nats on the second, so the model is near-perfect at in-context copying (Figure 1, top left).

Attention. The top five induction scores belong to heads 5.5 (0.94), 6.9 (0.92), 7.10 (0.92), 5.1 (0.91), 7.2 (0.84), which are exactly the induction heads previously reported for GPT-2 small. The next-best, 10.1, scores only 0.54. Head 4.11 is an almost pure previous-token head (prev = 0.99), with weaker ones at 3.7, 6.8, 2.2.

Patching single heads vs head sets.

patched	heads	r
single head	7.2	+0.140
single head	6.9	+0.133
single head	9.9	+0.107
single head (most negative)	10.7	−0.106
single head (most negative)	11.10	−0.072
previous-token head	4.11	+0.004
top-5 induction heads	5.5 6.9 7.10 5.1 7.2	+0.810
prev-token + top-5 induction	4.11 5.5 6.9 7.10 5.1 7.2	+0.816

Table 1: Fraction of the clean–corrupted gap recovered by patching head outputs.

Finding

The mechanism is **redundant**. Each induction head alone recovers only 10–14%, but the five together recover 81%. Single-head patching understates circuits spread over parallel heads. Two late heads have **negative** effects: 10.7 and 11.10. Head 10.7 is the copy-suppression head of McDougall et al. (2023), which down-weights tokens that earlier heads are pushing. Patching its clean behaviour in suppresses the right answer.

1.4 Where vs what: separating K-composition from copying

The previous-token head 4.11 barely registers in Table 1 ($r = +0.004$) despite a previous-token score of 0.99. To see why, we patch components at **first-copy positions only** and measure two quantities: the induction heads' attention to their target (the **where**, 0.00 in the corrupted run), and r (the prediction).

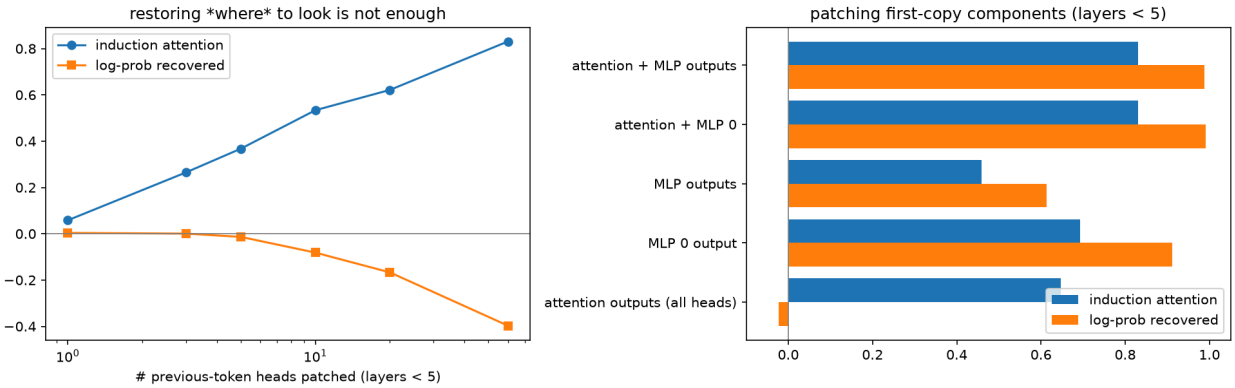


Figure 2: Left: patching the k strongest previous-token heads in layers 0–4. Attention to the induction target climbs steadily while r falls below zero. Right: patching whole components at first-copy positions in layers 0–4.

patched at first-copy positions	induction attention	r
top-1 previous-token heads	0.06	+0.004
top-3 previous-token heads	0.26	+0.001
top-5 previous-token heads	0.37	−0.014
top-10 previous-token heads	0.53	−0.081
top-20 previous-token heads	0.62	−0.167
top-60 previous-token heads	0.83	−0.397
attention outputs (all heads)	0.65	−0.023
MLP 0 output	0.69	+0.911
MLP outputs	0.46	+0.613
attention + MLP 0	0.83	+0.991
attention + MLP outputs	0.83	+0.989

Table 2: Where vs what. Clean-run induction attention is 0.90.

Finding

- **Previous-token information is spread over many heads.** Head 4.11 alone lifts induction attention from 0.00 to 0.06. The top 10 reach 0.53, and all 60 heads in layers 0–4 reach 0.83.
- **Restoring the where without the what hurts.** With all 60 heads patched, the induction heads attend correctly but copy the **corrupted** tokens b_k, and r drops to -0.40. This isolates the OV half of the circuit.
- **The what comes through MLP 0.** Patching MLP 0 alone gives $r = 0.91$, and restores attention too (0.69). This fits the known role of GPT-2's first MLP as an extended token embedding: downstream heads read the token identity from it. Attention plus MLP 0 gives 0.99.

Caveat

One seed, $L = 50$, 8 sequences, and random tokens drawn from ids 1000–20000. Random-token sequences are off-distribution for GPT-2, so the relative roles of the heads could shift on natural text. The next step is to repeat the analysis on repeated natural sentences and with resample-ablation.

2 Toy models of superposition (card 2.2)

Reproduces Elhage et al. (2022), *Toy Models of Superposition*. Code: `experiments/01_mech_interp/02_toy_superposition.py`. All parts ran on a Colab T4 through the MCP server: 17 s for (a), 47 s for (b), and 5 min for (c). The figures were redrawn locally from the saved JSON with `--replot`.

2.1 Model

n features are compressed into $m < n$ hidden dimensions and read back out with a tied weight matrix $W \in \mathbb{R}^{m \times n}$:

$$h = Wx, \quad x' = \mathrm{ReLU}(W^\top h + b), \quad \mathcal{L} = \mathbb{E}_x \sum_{i=1}^{n} I_i (x_i - x'_i)^2. \tag{4}$$

Each feature is present independently with probability $p = 1 - S$ (the **density**; S is the **sparsity**), and then $x_i \sim U[0, 1]$. I_i is its **importance**. Expanding coordinate i shows where the tension comes from:

$$x'_i = \mathrm{ReLU}\left(\|W_i\|^2 x_i + \sum_{j \neq i} (W_i \cdot W_j) x_j + b_i \right), \tag{5}$$

where W_i is column i. With $m < n$ the columns cannot all be orthogonal, so representing more than m features costs **interference** $(W_i \cdot W_j)x_j$. When features are sparse, two rarely occur together, so the interference term is usually zero. The ReLU with a negative bias removes the small positive interference that remains. So sparsity should make superposition worth its cost. All experiments train many small models in parallel (Adam, learning rate 10^{-3} with cosine decay, batch 1024) and keep the best of several seeds.

2.2 (a) Geometry: 5 features in 2 dimensions

With $n = 5$, $m = 2$ and $I_i = 0.9^i$:

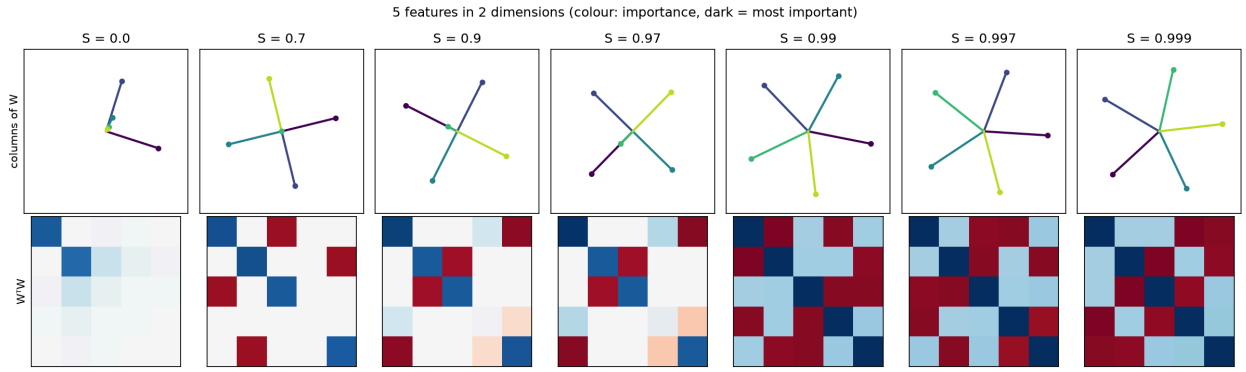


Figure 3: Top: columns $W_i \in \mathbb{R}^2$ (dark = most important). Bottom: $W^\top W$ (blue positive, red negative). Features "represented" means $\|W_i\| > 0.5$.

S	0	0.7	0.9	0.97	0.99	0.997	0.999
features represented	2	4	4	4	5	5	5
loss	0.180	0.098	0.024	0.007	0.001	0.000	0.000

Table 3: Represented features and final loss for the best of 4 seeds per sparsity.

Finding

Three regimes, as in the paper:

- **Dense** ($S = 0$): the textbook PCA-like solution. The 2 most important features take orthogonal directions and the rest are dropped.
- **Intermediate** ($S = 0.7$–0.97): four features form two **antipodal pairs**, an "X" whose $W^\top W$ has -1 entries.
- **Sparse** ($S \geq 0.99$): all five sit on a regular **pentagon**, with norms rising to about 1.16 and a negative bias of about -0.27. The bias is the ReLU threshold that clips the positive interference in Equation 5.

Represented features per sparsity: $2 \to 4 \to 4 \to 4 \to 5 \to 5 \to 5$.

Caveat

The dense and sparse ends were identical across three independent runs. The count in the transition zone ($S = 0.7$–0.97) was not: it ranged from 3 to 5 depending on the seeds, because several configurations reach nearly equal loss there. The feature dropped in the intermediate regime is #3 rather than the least important #4. Their importances (0.9^3 and 0.9^4) differ by only 10%, so which one loses is close to a coin flip.

2.3 (b) Phase diagram: 2 features in 1 dimension

With $n = 2$ and $m = 1$, $W = (w_1, w_2)$ is just two numbers. Feature 1 has importance 1 and feature 2 has importance I_2. We sweep $I_2 \in [0.1, 10]$ and the density $1 - S \in [0.01, 1]$ on a 24×24 log grid, with 4 seeds per cell, and classify the best model by which $|w_i| > 0.5$.

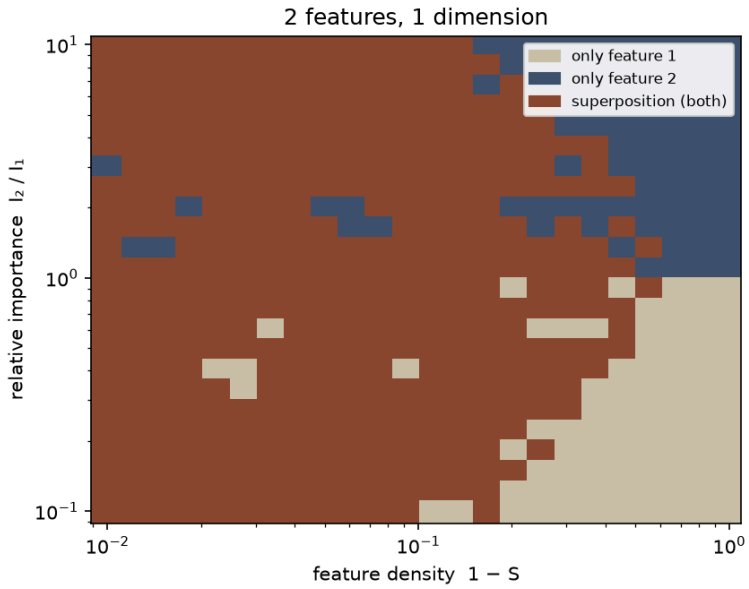


Figure 4: Which features the 1-dimensional model represents, as a function of density and relative importance.

Finding

Three phases, as in the paper (cells: only feature 1: 86, only feature 2: 88, both: 402):

- With **dense** features the model is a hard switch at $I_2/I_1 = 1$: it keeps whichever feature is more important.
- As density falls, superposition takes over. The more unequal the importances, the sparser the features must be first: from about $1 - S \approx 0.5$ at equal importance to about 0.1–0.2 at a 10× ratio. At very low density the model stores both features even when one matters 10× less, because they almost never co-occur.
- **Every** superposed solution is antipodal (100%, $w_1 w_2 < 0$). With opposite signs, the interference when one feature is active is negative, and the ReLU clips it to zero at no cost.

Caveat

The isolated off-phase cells deep inside the superposition region are optimisation noise, not phases. There, every loss scales with the density, so the gap between configurations is tiny (about 10^{-3}) relative to the minibatch noise of 4 seeds × 10^4 steps.

2.4 (c) How many dimensions does a feature get?

For $n = 200$ features in $m = 20$ dimensions with uniform importance, two measures:

$$D^* = \frac{m}{\|W\|_F^2} \quad \text{(dimensions per represented feature)}, \quad D_i = \frac{\|W_i\|^2}{\sum_j \left(\widehat{W}_i \cdot W_j\right)^2}, \tag{6}$$

where $\widehat{W}_i = \frac{W_i}{\|W_i\|}$. D_i is the fraction of a dimension that feature i has to itself: 1 for a dedicated direction, $\frac{1}{2}$ for each member of an antipodal pair, $\frac{2}{3}$ for a triangle, $\frac{2}{5}$ for a pentagon, $\frac{3}{4}$ for a tetrahedron and $\frac{3}{8}$ for a square antiprism.

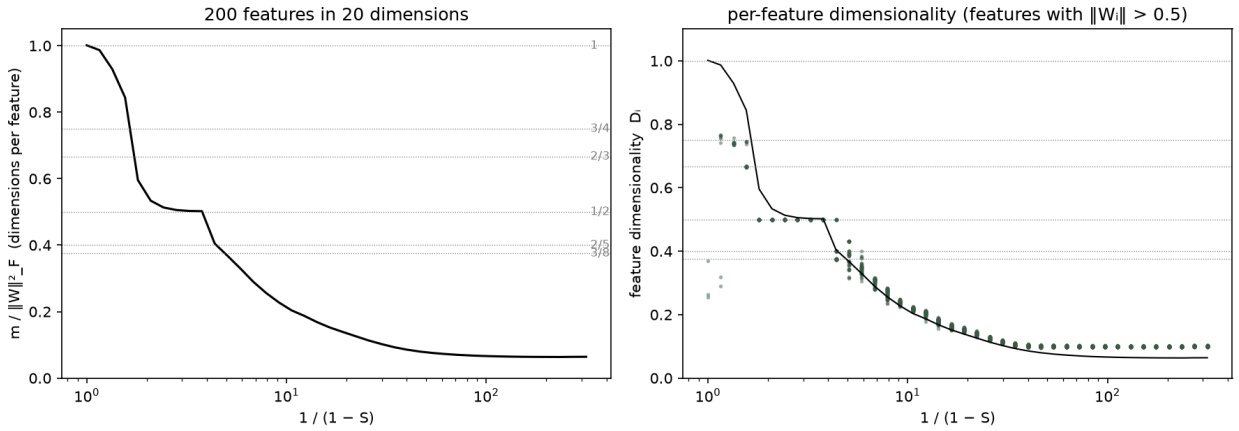


Figure 5: Left: D^* against $\frac{1}{1-S}$; dotted lines mark the configuration fractions. Right: D_i for every represented feature at each sparsity.

Finding

The curve is not smooth. It has the regimes the paper describes:

- **Dense** ($S = 0$): $D^* = 1.00$. The model uses all m dimensions, but with uniform importance it learns a rank-m projection spread thinly across many features, as PCA would. Only 3 features reach norm 0.5.
- **Low sparsity** ($1/(1-S) \approx 1.2$–1.6): of the 54 represented features, 31 sit at $D_i \approx \frac{3}{4}$ (tetrahedra) and 21 at $\approx \frac{2}{3}$ (triangles).
- **A plateau at exactly $\frac{1}{2}$** for $1/(1-S) \in [2.42, 3.78]$: in those 4 models, 152 of 152 represented features have $|D_i - \frac{1}{2}| < 0.01$. Every feature is in an antipodal pair.
- **Sparse**: after brief clusters near $\frac{2}{5}$ and $\frac{3}{8}$, D_i decreases continuously until 199 of the 200 features are represented, with $D_i = 0.100 = m/n$.

Over the whole sweep only 8% of feature–sparsity points lie within 0.03 of a configuration value (3/4: 31, 2/3: 21, 1/2: 238, 2/5: 31, 3/8: 64), because the sparse end is a continuum. Where the model is still **choosing** a geometry, it chooses discrete ones.

Caveat

Part (c) is one seed per sparsity at 50000 steps and $n = 200$, versus the paper's larger models. The sticky points beyond $\frac{1}{2}$ (such as $\frac{2}{5}$ and $\frac{3}{8}$) are therefore only transient here. A longer run with $n = 400$ would show whether they widen into plateaus. The earlier 10^4-step run found the same $\frac{1}{2}$ plateau but blurrier edges.

3 Remaining cards

DONE 2.1 Induction heads: attention scores, per-head and head-set patching, where-vs-what decomposition.

DONE 2.2 Toy models of superposition: geometry, phase diagram, feature dimensionality.

IN PROGRESS 2.3 Train an SAE on one GPT-2 small layer and interpret 5 features. Training on a Colab T4; section to follow.

TO DO 2.4 *Original*: hand-trace one single-token completion circuit with activation/path patching. This is your card; the tooling in 2.1 is the starting point.